



Protocol Audit Report

Version 1.0

Cyfrin.io

July 6, 2026

Protocol Audit Report

ALtera21

July 6, 2026

Prepared by: Cyfrin Security Researcher: ALtera21

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing user to receive the incorrect amount of tokens
 - [H-4] `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of $x * y = k$

- Medium
 - [M-1] `TSwapPool::deposit` is missing deadline checks causing transactions to complete even after the deadline
 - [M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

Protocol Summary

A smart contract application for gambling a random rarity picture of puppies.

Disclaimer

The “I do it myself” team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash: 7bd52c6a75f1115b23e0cff0fa16f7c522703fcd

Scope

```
1 ./src/  
2 #-- PoolFactory.sol  
3 #-- TSwapPool.sol
```

Roles

- LiquidityProvider: give liquidity to the pool by calling `deposit` function and receive LP tokens as proof of ownership of the liquidity. had the power to withdraw the liquidity in exchange for LP tokens.
- swapper/users: can swap their desired token for WETH or vice versa by calling either `swapExactInput` or `swapExactOutput`, can also sell their tokens for WETH by calling `sellPoolTokens` function

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	2
Low	0
Info	0
Total	0

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of output tokens. However, the function currently miscalculate the resulting amount. When calculating the fee, it scales the amount by `10_000` instead of `1_000`.

Impact: Protocol takes more fees than expected from users.

Proof of Concept:

Add the following test function inside `test/unit/TSwapPool.t.sol` test suite

test code

```
1      function testSwapExactOutputMiscalculateFees() public {
2
3          poolToken.mint(address(pool), 100e18);
4          weth.mint(address(pool), 10e18);
5
6          uint256 poolTokenOutputAmountWithNoFee = 1e18;
7          uint256 supposedWethToPaidWithNoFee = 1e17;
8
9          uint256 poolTokenBefore = poolToken.balanceOf(user); // should
           have 10e18
10         uint256 wethBefore = weth.balanceOf(user); // should have 10e18
11
12         vm.startPrank(user);
13         weth.approve(address(pool), type(uint256).max);
14         pool.swapExactOutput(IERC20(weth), IERC20(poolToken),
           poolTokenOutputAmountWithNoFee, uint64(block.timestamp));
15         vm.stopPrank();
16
17         uint256 poolTokenAfter = poolToken.balanceOf(user); // should
           have 10e18 + 1e18
18         uint256 wethAfter = weth.balanceOf(user); // should be 10e18 - 1
           e17, it should be.
19
20         uint256 expectedFeePaid = poolTokenOutputAmountWithNoFee - pool
           .getOutputAmountBasedOnInput(supposedWethToPaidWithNoFee,
           weth.balanceOf(address(pool)), poolToken.balanceOf(address(
           pool)));
```

```
21         uint256 actualFeePaid = wethBefore - wethAfter -
           supposedWethToPaidWithNoFee;
22
23         assertGt(actualFeePaid, expectedFeePaid);
24
25         console.log("actual fee paid", actualFeePaid);
26         console.log("expected fee paid", expectedFeePaid);
27
28         console.log(poolTokenBefore);
29         console.log(wethBefore);
30         console.log(poolTokenAfter);
31         console.log(wethAfter);
32     }
```

Recommended Mitigation:

```
1     function getInputAmountBasedOnOutput(
2         uint256 outputAmount,
3         uint256 inputReserves,
4         uint256 outputReserves
5     )
6     public
7     pure
8     revertIfZero(outputAmount)
9     revertIfZero(outputReserves)
10    returns (uint256 inputAmount)
11    {
12        return
13        -      (((inputReserves * outputAmount) * 10000) /
14        +      (((inputReserves * outputAmount) * 1000) /
15              ((outputReserves - outputAmount) * 997));
16    }
```

[H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap

Proof of Concept:

Consider the following scenario: 1. The price of WETH is 1_000 USDC 2. user input a `swapExactOutput` looking for 1 WETH 3. inputToken = USDC 4. outputToken = WETH 5. outputAmount = 1 WETH 6.

deadline = whatever 3. The function does not offer `maxInputAmount` 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 weth is now 10_000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10_000 USDC, instead of the expected 1_000 USDC

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol (IMPORTANT* SOME COULD ARGUE THAT WE COULD JUST DON'T APPROVE TOO MANY TOKENS TO LIMIT IT);

```
1      function swapExactOutput(  
2          IERC20 inputToken,  
3      +      uint256 maxInputAmount,  
4      .  
5      .  
6      .  
7          inputAmount = getInputAmountBasedOnOutput(  
8              outputAmount,  
9              inputReserves,  
10             outputReserves  
11         );  
12  
13 +     if (inputAmount > maxInputAmount) {  
14 +         revert();  
15 +     }  
16  
17     _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] TSwapPool::sellPoolTokens mismatches input and output tokens causing user to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

Add the following test function inside `test/unit/TSwapPool.t.sol` test suite

test code

```
1     function testsellPoolTokensIsWrong() public {
2         poolToken.mint(address(pool), 100e18);
3         weth.mint(address(pool), 10e18);
4
5         uint256 poolTokenBefore = poolToken.balanceOf(user); // should
           have 10e18
6         uint256 wethBefore = weth.balanceOf(user); // should have 10e18
7
8         uint256 poolTokenToSell = 1e10;
9
10        vm.startPrank(user);
11        weth.approve(address(pool), type(uint256).max);
12        poolToken.approve(address(pool), type(uint256).max);
13        pool.sellPoolTokens(poolTokenToSell);
14        vm.stopPrank();
15
16        vm.expectRevert();
17        assertEq(poolTokenBefore, 0); // the pool token should be zero
18    }
```

Recommended Mitigation:

Consider Changing the implementation to use `swapExactInput` instead of `swapExactOutput`
Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (i.e. `minWethToReceive` to be passed to `swapExactInput`)

```
1     function sellPoolTokens(
2         uint256 poolTokenAmount,
3 +         uint256 minWethToReceive
4     ) external returns (uint256 wethAmount) {
5         return
6 +         swapExactInput(
7 +             i_poolToken,
8 +             poolTokenAmount,
9 +             i_wethToken,
10 +             minWethToReceive,
11 +             uint64(block.timestamp)
12 +         );
13 -         swapExactOutput(
14 -             i_poolToken,
15 -             i_wethToken,
16 -             poolTokenAmount,
17 -             uint64(block.timestamp)
18 -         );
19     }
```

Additionally it might be wise to add a deadline to the function, as the deadline is currently being defaulted to `block.timestamp` (MEV later)

[H-4] TSwapPool : : _swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$, Where: - x : The balance of the pool token - y : The balance of WETH - k : the constant product of the two balances

This means that whenever the balances change in the protocol, the ratio between the two amount should remain constant, hence the k . However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The following block of code is responsible for the issue

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1
5                                  _000_000_000_000_000_000); //! @audit, this is shit
6      }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentives given out by the protocol.

More simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentives of 1_000_000_000_000_000_000 tokens 2. That user continues to swap until all the protocol funds are drained

proof of code

Place the following into `TSwapPool.t.sol`

```
1      function testInvariantBroken() public {
2          vm.startPrank(liquidityProvider);
3          weth.approve(address(pool), 100e18);
4          poolToken.approve(address(pool), 100e18);
5          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6          vm.stopPrank();
7
8          uint256 outputWeth = 1e16;
9          // expectedDeltaX = int256(pool.
10             getPoolTokensToDepositBasedOnWeth(poolTokenAmountAdded));
11
12          vm.startPrank(user);
13          poolToken.approve(address(pool), type(uint256).max);
14          pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
15             timestamp));
16          pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
17             timestamp));
18          pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
19             timestamp));
```

```
16     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
17         timestamp));  
17     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
18         timestamp));  
18     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
19         timestamp));  
19     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
20         timestamp));  
20     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
21         timestamp));  
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
22         timestamp));  
22  
23     int256 startingY = int256(weth.balanceOf(address(pool)));  
24     int256 expectedDeltaY = int256(-1) * int256(outputWeth); // the  
25         minus 1 is there bcuz the pool is losing weth // the  
26         expected weth added is supposed to be outputWeth  
26  
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
27         timestamp));  
27     vm.stopPrank();  
28  
29     uint256 endingY = weth.balanceOf(address(pool));  
30  
31     int256 actualDeltaY = int256(endingY) - startingY;  
32  
33     assertEq(actualDeltaY, expectedDeltaY);  
34 }
```

Recommended Mitigation: Remove the extra incentives. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
1 -     swap_count++;  
2 -     if (swap_count >= SWAP_COUNT_MAX) {  
3 -         swap_count = 0;  
4 -         outputToken.safeTransfer(msg.sender, 1  
5 -             _000_000_000_000_000_000); //! @audit, this is shit  
5 -     }
```

Medium

[M-1] TSwapPool::deposit is missing deadline checks causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transactions to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: Consider making the following change to the function.

```
1     function deposit(  
2         uint256 wethToDeposit,  
3         uint256 minimumLiquidityTokensToMint,  
4         uint256 maximumPoolTokensToDeposit,  
5         uint64 deadline //! @audit param not used  
6     )  
7     external  
8 +     revertIfDeadlinePassed(deadline)  
9     revertIfZero(wethToDeposit)  
10    returns (uint256 liquidityTokensToMint)  
11    {  
12  
13        ...  
14  
15    }
```

[M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant